



A residual utility-based concept for high-utility itemset mining

Pushp Sra¹ · Satish Chand¹

Received: 1 January 2023 / Revised: 27 June 2023 / Accepted: 25 July 2023 /

Published online: 23 August 2023

© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2023

Abstract

Knowledge discovery in databases aims at finding useful information for decision-making. The problem of high-utility itemset mining (HUIM) has specifically garnered huge research attention, as it aims to find relevant information on patterns in a database, which conform to a user-defined utility function. The mined patterns are used for making data-backed decisions in the fields of healthcare, e-commerce, web analytics, etc. Various algorithms exist in the literature related to mining the high-utility items from the databases; however, most of them require multiple database scans, or deploy complex data structures. The utility-list is an efficient list-based data structure that is being widely adopted in the design of HUIM algorithms. The existing utility-list-based algorithms, however, suffer from some drawbacks like extensive use of inefficient join operations, multiple definitions of join operations, etc. Though the HUIM is an important research area, yet very little research has been directed towards improving the design of data structures used for the mining process. In this paper, we introduce the concept of residual utility to design two new data structures, called residue-map and master-map. Using these two data structures, a new algorithm, called R-Miner, is introduced for mining the high-utility items. In order to further optimise the mining process, the cumulative utility value is used as an upper bound and additional pruning conditions are also discussed. Several experiments are carried out on both real and synthetic datasets to compare the performance of R-Miner with the existing list-based algorithms. The experimental results show that the R-Miner improves the performance by up to the order of 2 as compared to the list-based algorithms: EFIM, H-Miner, HUI-Miner, FHM, and ULB-Miner.

Keywords High-utility itemset mining · Data mining · Knowledge discovery · Memory efficient mining

✉ Pushp Sra
pushpsmathur@gmail.com

Satish Chand
schand@mail.jnu.ac.in

¹ School of Computer and Systems Sciences, Jawaharlal Nehru University, New Delhi, India

1 Introduction

The techniques of data mining form the basis of knowledge discovery and are used to determine previously unknown, user relevant information from a database. One of the earliest approaches of data mining is the Frequent Itemset Mining (FIM) that finds the most frequently occurring items in a database and can be used in various applications such as market basket analysis, customer segmentation, and cross-marketing. The mining results generated by considering merely the frequency of an item in a database may not necessarily be very useful as FIM techniques do not consider other relevant parameters like quantity, or per unit profit of an item. For example, for a customer transaction database, the FIM outputs the set of most frequently occurring items, which may not be the most useful patterns for a person interested in finding the items that generate the maximum profit. The High-Utility Itemset Mining (HUIM) [25] addresses this problem by extracting patterns, based on a user-defined objective utility function. In the recent years, HUIM has become an important research area, but designing time and memory efficient algorithms for HUIM is still a challenging problem.

An important criterion for designing an efficient mining scheme for HUIM is the formation of search space. A naive approach generates all the combinations of the input items and checks the utility values of every combination against the utility threshold. For a database with n distinct items, this approach generates 2^n candidates, which can overwhelm the memory for large n . So, it is important to efficiently organise the search space in order to optimise the mining performance. The frequent itemset mining (FIM) approaches deploy a pruning condition, called the *downward closure property* [1], which states that all the subsets of a non-frequent itemset are also non-frequent. However, this property does not hold for HUIM as shown in [25], since a high-utility itemset (HUI) considers both, an internal and an external utility value. Therefore, the task of HUIM is more complex than FIM as the HUI's do not follow the monotonic or anti-monotonic property.

The second important consideration for designing an HUIM algorithm is the data structures used to store and retrieve the potential high utility itemsets. In the literature, several structures like arrays, lists and trees have been discussed to efficiently hold the utility information for the mining process. The utility lists were first defined in [16] and have garnered a huge attention because of their structural simplicity and the ease of recursive deployment for mining. A utility list is designed for each item in the database and stores information related to utility and the remaining utility of other items in the transaction. Further, join operation can be used to recursively produce utility lists of higher order by using utility lists of lower order.

Several variations of the utility lists have been proposed to improve the efficiency of mining process [5, 26]. In particular, a large amount of research has been devoted towards simplifying or reducing the number of join operations between the utility lists, which is the bottleneck of utility list-based algorithms. However, all of the improved algorithms make use of the same underlying utility list structure as defined originally [16] and very less research is directed towards the design of new and improved list-based data structures for utility mining. Another drawback of the utility list-based approach is that the utility lists are based on the concept of the remaining utility, which requires sorting of each transaction based on the transaction weighted utilisation value. For a database with a large number of transactions, the sorting process attributes significantly to the overall execution time.

The above-mentioned gaps necessitate the need to explore new designs for the list-based structures for high-utility itemset mining. This paper addresses these issues by introducing a new method to organise the search space and mine the relevant items. Our proposed work introduces two new data-structures, which are based on a novel concept of *residue*.

The main contributions of this paper are summarised as follows.

- Two new data structures, called the *residue map (RM)* and the *master-map*, are designed to store the utility-related information.
- A mining model called *R-Miner* is introduced that uses the residue maps to find the high-utility itemsets. Further, pruning conditions are proposed to reduce the search space.
- Several experiments are carried out on both real and synthetic datasets (including dense and sparse datasets) to show the efficacy of the proposed model R-Miner.

The remaining paper is organised as follows. In Sect. 2, the problem statement is discussed, and in Sect. 3, the existing techniques used for HUIM are reviewed. The proposed R-Miner algorithm is presented in Sect. 4, and the details of the experimental evaluation are provided in Sect. 5. The conclusion and the scope for future work are discussed in Sect. 6.

2 Problem definition

In this section, we discuss the problem of high-utility item-set mining. Consider a set of transactions, as shown in Fig. 1. This database consists of five transactions, where each transaction has a unique identifier, *TID*, and consists of pairs of item and its corresponding quantity in the transaction. The unit profit of each item is shown in Fig. 2. For a given item, the unit profit represents its external utility and the quantity is taken as the internal utility.

We introduce some definitions that are required to compute the utility of an item in a transaction and the transaction utility of a transaction (as shown in Fig. 3).

Definition 1 Utility: The *utility* of an item in a transaction is defined as the product of its external utility and the internal utility in that transaction.

Definition 2 Transaction Utility: The *transaction utility* is the sum of the utilities of all items in that transaction.

TID	Transaction
1	3:1, 5:1 , 1:1, 2:5, 4:3, 6:1
2	3:3, 5:1, 2:4, 4:3
3	3:1, 1:1, 4:1
4	3:6, 5:2, 1:2, 7:5
5	3:2, 5:1, 2:2, 7:2

Fig. 1 Transaction Database

Item	Profit
1	5
2	2
3	1
4	2
5	3
6	5
7	1

Fig. 2 Profit Table

TID	Item:Utility	Transaction Utility
1	3:1, 5:3, 1:5, 2:10, 4:6, 6:5	30
2	3:3, 5:3, 2:8, 4:6	20
3	3:1, 1:5, 4:2	8
4	3:6, 5:6, 1:10, 7:5	27
5	3:2, 5:3, 2:4, 7:2	11

Fig. 3 Transaction utility-based view of transactions

Itemset	Utility
2,4	30
2,5	31
2,3,4	34
1,3,5	31
2,3,5	37
2,4,5	36
2,3,4,5	40
1,2,3,4,5,6	30

Fig. 4 High-Utility Itemsets

Definition 3 Total Utility: The *total utility* of an item in a database is computed as the sum of its utilities across all the transactions, in which the item occurs.

Definition 4 High-Utility Itemset(HUI) If the total utility of an itemset is higher than the user-specified utility threshold, then it is called a *High-Utility Itemset(HUI)*.

For the database given in Fig. 3, the sets of HUIs for the threshold of 30 are provided in Fig. 4.

The problem of HUIM can therefore be formally stated as follows. The terminologies used here are adopted from the work discussed in [17].

- $I = \{i_1, i_2, \dots, i_m\}$ represents a set of items.
- $D = \{T_1, T_2, \dots, T_n\}$ is a database of transactions such that every transaction $T_i \in D$, consists of pairs of the form $a:b$, where a is the item-id and b is the internal utility value of a .
- $q(i_p, T_q)$ represents the internal or local transaction utility value. It is the quantity of item i_p in transaction T_q .
- $pr(i_p)$ is the external utility, which is the profit associated with item i_p .
- $u(i_p, T_q)$ is the measure of utility for item i_p in transaction T_q . It is defined as $q(i_p, T_q) \times pr(i_p)$.
- $u(X, T_q)$ is the utility of itemset X in transaction T_q . It is defined as $\sum_{i_p \in X} u(i_p, T_q)$, where $X = \{i_1, i_2, \dots, i_r\}$ is a r -itemset, $X \subseteq T_q$ and $1 \leq r \leq m$, and $\{i_1, i_2, \dots, i_m\}$ represents the set of items in D .
- $u(X)$ represents the utility of itemset X in the database D , which is $\sum_{T_q \in D \wedge X \subseteq T_q} u(X, T_q)$.

The problem of utility mining is to mine all the high-utility itemsets in a given database. An itemset X is a high-utility itemset if $u(X) \geq \epsilon$, where $X \subseteq I$ and ϵ is the minimum utility threshold.

3 Related work

In this section, we review the existing techniques for HUIM in detail.

3.1 Mathematical approaches

The problem of HUIM was first defined using a predictive model based on mathematical properties, called mining with expected utility (*MEU*) [25]. MEU model estimates the set of HUIs by forming potential candidate item-sets using a heuristic. The main drawback of this approach is that it fails to produce the exact results and also, overestimates the utility values. The mathematical conditions of the utility values are also analysed in [24]. Such predictive techniques lack scalability due to the generation of a huge number of candidates.

3.2 Generate-and-test-based techniques

These techniques are based on the level-by-level candidate generate and test approach, similar to the widely adopted a priori algorithm [1] for FIM. The a priori algorithm uses the *downward closure property* for efficiently pruning the search space. For the problem of utility mining, the Transaction-Weighted Downward Closure (TWDC) property is defined in the Two-Phase algorithm [17]. The TWDC represents an upper bound value on the maximum utility and is primarily used as the first step of mining in many high-utility mining algorithms [23].

3.3 Tree-based pattern growth techniques

The tree-based techniques for HUIM are designed to mitigate the formation of a huge search space, as in the case of the generate and test techniques. The tree-based techniques organise the search space using a compact representation of a tree structure to ensure efficient utilisation of the memory. The FP-Growth [9] algorithm for FIM is used as the baseline for several tree-based utility mining algorithms. The IHUP-Tree [2] provides a solution for incremental utility mining, in which each node of IHUP tree stores item id, support, and TWU values. An ordering is imposed on the itemsets to retain the sorted order during the mining process. The design of Utility Pattern Tree (UP-Tree) [22] is proposed to reduce the generation of candidate item-sets. The algorithm constructs a global UP-tree by using the local optimisations in order to discard the unpromising candidates by decrementing the utility values of the nodes. The UP Growth+ [21] reduces the number of candidates by decreasing the overestimated utility values. The pre-large-based incremental high-utility pattern mining for transaction modification (PIHUP-MOD) [27] algorithm uses the tree data structure for handling the transaction modifications, based on the pre-large concept in which the large and pre-large patterns are maintained.

3.4 List-based techniques

The HUI-Miner [16] is one of the most efficient algorithms that achieves time optimisation by using the utility lists. It constructs a set enumeration tree to optimally generate the search space by using a compact data structure, called utility list, that stores the utility value and pruning information. However, the main drawback of HUI-Miner is the costly join operation that performs intersection of the utility lists to obtain the lists of larger itemsets. Several

ItemSet	TID-list	Total Utility	Residue
X	$T_i : U_i, T_j : U_j, \dots T_n : U_n$	$\sum_{k=i}^n U_i$	$MinUtil - \sum_{k=i}^n U_i$

Fig. 5 Representation of a Residue Map

optimisations are proposed [7, 12, 13] to minimise the overhead of the costly join operations. Paper [28] introduces an algorithm, called EFIM, which relies on several upper-bounds to efficiently prune the search space. However, EFIM algorithm works well for dense datasets only. The ULB-miner algorithm [5] introduces a concept of buffering of the itemsets in order to optimise the mining process to improve the performance over HUI-Miner.

Several techniques are proposed in the literature to address the problem of HUIM; however, most of them require an extensive manipulation of the underlying data structures and lack of generalisation, which leads to complex mining mechanisms. For example, the HUI-Miner algorithm [16] calculates the utility values of single itemsets and itemsets with length more than one differently. The EFIM algorithm [28] involves multiple procedures for computation of the bounds and also uses several intermediate data structures.

Several variations of the high-utility itemset mining use the above-mentioned approaches as a baseline-like mining from incremental datasets or data streams [11], average utility mining [14], HUIM for negative utility values [4, 10], mining of HUI's from noisy environments [3] and mining from uncertain databases [15, 19]. New variations of HUIM have also been introduced, like the problem of utility occupancy pattern mining [8, 20], which considers the proportion of the pattern in a transaction, in addition to its quantity and unit profit.

Though the research related to HUIM has expanded, yet very few baseline algorithms exist to tackle the challenge of efficiently mining the patterns from the large datasets. This necessitates the need of devising better data storage and mining strategies. In order to address these challenges, we propose the *R-Miner* algorithm in the next section.

4 Proposed work

In this section, we introduce the proposed technique for mining the high-utility itemsets. We first introduce the data structures that are used for storing and retrieving the high-utility items and then, discuss the algorithm.

4.1 Data structures

1. Residue Map

The main data structure used in the proposed R-Miner algorithm to store the utility information is the residue map (RM), which is defined below.

Definition 5 *Residue map* is a structure that maps an itemset to the Transaction ID list, total utility, and residue value, as shown in Fig. 5. The *itemset* field stores the item identifier, and the *TID-list* stores the pair of transaction ID and corresponding utility value of the itemset in transaction. The residue map of an item i is represented as RM_i .

Definition 6 *Total utility* of a residue map is defined as the sum of the utility values of that itemset in the database.

ItemSet	TID-list	Total Utility	Residue
1	1:5, 3:5, 4:10	20	10
2	1:10, 2:8, 5:4	22	8
3	1:1, 2:3, 3:1, 4:6, 5:2	13	17
4	1:6, 2:6, 3:2	14	16
5	1:3, 2:3, 4:6, 5:3	15	15
6	1:5	5	25
7	4:5, 5:2	7	23

Fig. 6 Representation of the set of Residue Maps with cardinality one

ItemSet	TID-list	Total Utility	Residue
3,6	1:6	6	24
4,6	1:11	11	19
1,3	1:6, 3:6, 4:16	28	2
1,3,6	1:12	12	18

Fig. 7 Representation of the set of Residue Maps with cardinality greater than one

Definition 7 *Residue* represents the residual utility value required by an itemset to be considered a high-utility itemset. It is calculated by taking the difference between the minimum utility threshold and the total utility value. It should be noted that the residue value can hold a negative value if an itemset has the total utility value higher than the threshold.

Definition 8 *Cardinality* is defined as the size of the itemset of a residue map, i.e. the number of distinct elements present in the itemset of a residue map.

Example 1 Formation of residue maps

Fig. 6 provides the set of residue maps of cardinality one for the database in Fig. 3. Each row of Fig. 6 represents a residue-map for single items, assuming the minimum utility threshold as 30.

The residue maps of itemsets having cardinality greater than one can be constructed directly by using the residue maps of the itemsets with cardinality one. For this example, the residue map of the itemset $RM_{3,6}$ shown in Fig. 7 is constructed by using the residue maps RM_3 and RM_6 . This is done by first taking the intersection of the TID-lists of RM_3 and RM_6 and forming TID-list for the residue map $RM_{3,6}$. The intersection is performed by taking the set of transactions common in both TID-lists and adding their corresponding utility values. The total utility is then computed by taking the sum of the individual utility values from the newly formed TID-list. Finally, the residue is computed as the difference between the minimum utility threshold and the total utility value. Similarly, the residue maps of higher cardinalities can be computed directly from the residue maps of lower cardinality.

2. Master Map

Definition 9 *Master Map*: A master map is a linked list like structure that stores the pointers to all the residue maps of cardinality one in database, as shown in Fig. 8.

A master map consists of two fields: an identifier representing the item, and a pointer to the residue map of the item. An ordering, represented by $\succ$, is imposed on the master map,

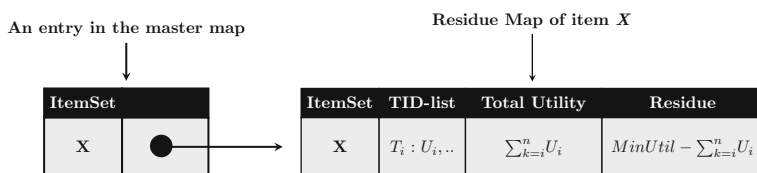


Fig. 8 Representation of a Master Map

ItemSet		ItemSet	TID-list	Total Utility	Residue
6	●	6	1:5	5	25
7	●	7	4:5, 5:2	7	23
3	●	3	1:1, 2:3, 3:1, 4:6, 5:2	13	17
4	●	4	1:6, 2:6, 3:2	14	16
5	●	5	1:3, 2:3, 4:6, 5:3	15	15
1	●	1	1:5, 3:5, 4:10	20	10
2	●	2	1:10, 2:8, 5:4	22	8

Fig. 9 Master Map

which is the total utility based on ascending order of the residue maps. The master map based on the sorted order of residue maps of Fig. 6 is shown in Fig. 9. It should be noted that sorting the set of residue maps in ascending order of the total utility automatically sorts them in the descending order of the residue values. This is an important design feature of the master map, which are used as the pruning conditions for the mining process.

4.2 The R-Miner algorithm

In this section, we present the R-Miner, for which the main procedure is provided in Algorithm 1. The R-Miner algorithm requires only a single database scan to construct the initial set of residue maps of cardinality one, based on which the master map is constructed. The high-utility itemsets are then mined using these two structures. The Algorithm 1 takes the transaction database and the user-defined minimum utility threshold as the input and produces the final set of high-utility items in the database as an output. We first construct the residue maps of cardinality one as follows. For the first occurrence of an item i , a TID-list is created to store the pairs of TID and utility values (line: 2–3). The total utility field stores the utility value of the item in the current transaction. The residue is computed by taking the difference between the user defined minimum utility threshold and the total utility value (line: 4–5). The residue map for the item is formed using the computed TID-list, total utility value, and the residue (line: 7). Finally, the master map is updated to contain a reference to the newly constructed residue-map (line: 8). The transaction weighted utility (TWU) value of the master map is initialized with the transaction utility value of the current transaction, where the transaction utility is defined as the sum of the utility values of all the itemsets in the current transaction.

If an item has already been scanned previously, then the residue map of that item already exists. The existing residue map for the item is retrieved using the master map (line: 11). The

TID-list is updated to include the current transaction ID and the corresponding utility value (line: 12–13). The values of the total utility and the residue are updated based on the TID-list (line: 16). The TWU value is updated to include the transaction utility value of the current transaction (line: 17). Since a reference to this residue-map already exists in the master-map, the master-map is not required to be updated again. After the initial scan of the database, the construction of the master map and the residue maps of cardinality one is complete. A total ordering, $\succ$, which is the ascending order of the total utility values of the residue-maps, is then imposed on the master-map (line: 20). Only those residue maps are included in the sorted master map, which have the transaction weighted utility (TWU) value higher than the minimum utility threshold. Then, R-Miner (given in Algorithm 2) is called for every residue-map in the sorted master-map (line: 22 - 25). The index of the current residue map and the maximum size of the sorted master-map are also passed as arguments to the function call.

Algorithm 1 Main Procedure

INPUT:

D : Database with k Transactions $(T_1, T_2, T_3 \dots T_k)$ and n distinct items. Each item is assigned a utility value U_i .

$MinUtil$: Minimum utility Threshold

OUTPUT:

HUI -set: Set of high-utility items

```

for each:  $T_i \in D$ 
  for each:  $I \in T_i$ 
1: if  $I \notin M$  then
2:    $TL \leftarrow$  new TID-list
3:    $TL \leftarrow (T_i, U_i)$ 
4:    $TotUtil(TU_i) \leftarrow U_i$ 
5:    $Residue(R) \leftarrow MinUtil - TotUtil$ 
6:    $rMap \leftarrow$  new ResidueMap
7:    $rMap(I, TL, TU_i, R)$ 
8:    $M \leftarrow M(rMap)$ 
9:    $TWU_i \leftarrow TU_i$ 
10: else
11:    $rMap \leftarrow$  get ResidueMap( $I$ )
12:    $TL \leftarrow$  get TID-list ( $rMap$ )
13:   update  $TL(T_i, U_i)$ 
14:   update  $TotUtil(TU_i) \leftarrow \sum U_i$ 
15:   update Residue( $R$ )
16:   update  $rMap(I, TL, TU_i, R)$ 
17:    $TWU_i \leftarrow TWU_i + TU_i$ 
18: end if
19: if  $TWU_i > MinUtil$  then
20:    $SM \leftarrow M \succ (U_i)$ 
21: end if
22: for all  $rMap_i \in SM$  do
23:    $HUI_i \leftarrow R\text{-Miner}(rMap_i, SM, MinUtil, i, sizeof(SM))$ 
24:    $HUI\text{-set} \leftarrow HUI\text{-set} \cup HUI_i$ 
25: end for
26: return ( $HUI\text{-set}$ )
  
```

The mining procedure is recursively called to extract all the combinations of high-utility itemsets based on the following conditions.

Condition 1 *Condition to compute Join*: The join of a residue map RM_i is computed with another residue map RM_j only if the *total utility* value of RM_j is higher than or equal to the *residue* value of RM_i .

Condition 2 *Ordering for search space exploration*: In order to ensure an optimal number of join operations for constructing a residue map of cardinality n from a residue map of cardinality $n-1$, the master map is traversed in the reverse order.

Additionally, the R-Miner algorithm uses the following two pruning conditions:

Pruning Condition 1 *Total utility as an upper bound for cardinality one*: If the residue of the current residue map is higher than the total utility of the last element of the master map, then the join operation need not be performed for the current residue map.

Pruning Condition 2 *Maximum total utility as upper bound*: The maximum total utility in the current set of residue maps can be used as an upper bound for pruning. If the residue value of a map is higher than the maximum total utility in the current set of residue maps, it implies that no residue map can satisfy the utility requirement for the residue map, and hence, the residue map should not be explored any further.

Pruning Condition 3 *Negative Residue*: If the residue value of a residue map is negative, it implies that the residue map qualifies for a high-utility itemset. Hence, it can be produced as an output directly.

The R-Miner (Algorithm 2) is a recursive procedure, which takes a residue-map as input and computes the join operations based on Condition-1 and Condition-2. It produces the set of HUIs for the given residue-map as an output.

Working Principle The R-Miner algorithm works on the principle of generation of the search space by forming residue maps of higher cardinalities based on the residue maps having lower cardinality. In order to generate a residue map of cardinality n , a join operation of the residue map with cardinality $n-1$ is performed with a residue map of cardinality 1 .

$$RM_n \leftarrow RM_{n-1} \cap RM_1$$

For an input residue map, the R-Miner first iterates over the master map and computes the join operation with each element of the master map based on Condition-1 (line: 8). If the computed join operation is not null, then the total utility of the newly formed residue map is checked against the minimum utility threshold (line: 11). If the total utility crosses the threshold, then the itemset is produced as an output. The R-Miner algorithm is then recursively called with a decremented value of the current index (line: 14). If the condition for computing the join (line: 8) is not satisfied, then no further join operations are explored (line: 17) for the residue map.

The computation required for the join operation between two input residue maps is provided in Algorithm 3. First, an intersection of the itemsets of two input residue maps is computed. If the intersection is not null, then new TID-list is formed; otherwise, a null value is returned to the calling procedure (line: 3). If a common TID is present (line: 12) in both ID-lists, then the TID-list of the new residue map is formed using the pair of TID and sum of utility values from the corresponding residue maps (line: 15–16). The total utility and the residue value are computed from the newly formed TID-list (line: 17–18). The newly computed residue map is returned to the calling procedure (line: 23).

Example 2 Consider the set of residue maps in Fig. 9. It can be observed that the residue map RM_6 has the least value for total utility, which is 5. As discussed previously, the residue essentially represents the utility required for the itemset to qualify as a high-utility itemset. For the residue map RM_6 , the residue of 25 implies that the residue maps offering a minimum

Algorithm 2 R-Miner

```

INPUT:
rMapi: input residue-map
SM: master-map
MinUtil: Minimum utility Threshold
cIndx: current index of the input residue-map
outIndx: size of master-map
OUTPUT:
HUIs: Set of high utility itemsets

1: if outIndx < cIndx then
2:   return
3: end if
4: outIndx ← SM(outIndx)
5: while outIndx.hasPrevious() do
6:   outIndx ←
7:   rMapj ← outIndx.Previous()
8:   if rMapj.totUtil ≥ rMapj.Residue then
9:     Join ← ComputeJoin(rMapj, rMapj)
10:    if Join != null then
11:      if Join.totUtil ≥ MinUtil then
12:        HUI ← Join.itemSet
13:      end if
14:      R-Miner(Join, SM, MinUtil, outIndx, cIndx--)
15:    end if
16:  else
17:    break
18:  end if
19: end while
20: return(HUI)

```

utility of 25 can yield meaningful results after the join operation, which follows Condition 1. Since the master map is sorted in increasing order of utility values, the residue maps with the highest values are pushed towards the bottom of the master-map. In order ensure that only meaningful join operations are done for a residue-map, the master map is first traversed from the bottom. This is done to check for the elements which have total utility value greater than the residue of that map (which follows Condition 2). For the residue map RM_6 with utility 5 and residue 25, by traversing the master-map from the end, the first element encountered is RM_2 with total utility of 22, which is less than the desired value of 25. So, the join operation is not performed. Since the master-map is sorted in ascending order, the total utility value of all the residue maps before RM_2 shall be less than 22, and hence, these need not be considered any further for performing the join (Pruning Condition 1). Similarly, for the residue map RM_7 , the residue is 23 and the total utility of the last element in master map is 22, and hence, no join operations needs be performed. For the residue map RM_3 , the residue is 12, the join operation is performed with the residue maps RM_2 and RM_1 that hold total utility values of 22 and 20, respectively. Similar procedure is followed for the remaining residue-maps.

4.3 Execution time analysis

For a database with n transactions and m distinct items, the execution time analysis of R-Miner can be divided into the following two steps:

1. Scanning of database and formation of master map:

The set of single cardinality residue maps are formed in first scan of the database and it takes $O(n)$ time. For m distinct items, a maximum of m residue maps shall be constructed. For the worst case analysis, all the m residue maps have the transaction weighted utilisation

Algorithm 3 Compute Join

```

INPUT:
 $rMap_i$ : input residue-map( $i$ )
 $rMap_j$ : input residue-map( $j$ )
OUTPUT:
 $rMap$ : intersection of  $rMap_i$  and  $rMap_j$ 

1: newItemSet  $\leftarrow rMap_i.itemSet \cap rMap_j.itemSet$ 
2: if newItemSet == null then
3:   return
4: end if
5: TotUtil( $TU_i$ )  $\leftarrow 0$ 
6: size  $\leftarrow \text{minimum}(rMap_i.size(), rMap_j.size())$ 
7:  $TL_i \leftarrow \text{TID-list}_i$ 
8:  $TL_j \leftarrow \text{TID-list}_j$ 
9:  $i \leftarrow 1$ 
10: for all  $i \leq \text{size}$  do
11:    $element_i \leftarrow \text{getElement}(TL_i, i)$ 
12:   searchindex  $\leftarrow \text{Search}(TL_j, element_i.tid)$ 
13:   if searchindex != null then
14:      $element_j \leftarrow \text{getElement}(TL_j, searchindex)$ 
15:     newUtil  $\leftarrow element_i.iUtil + element_j.iUtil$ 
16:     new $T_L \leftarrow element_i.tid, newUtil$ 
17:      $T \leftarrow T + newUtil$ 
18:      $R \leftarrow \text{MinUtil} - T$ 
19:   end if
20: end for
21:  $rMap \leftarrow \text{new ResidueMap}$ 
22:  $rMap(\text{newItemSet}, \text{new}_{T_L}, T, R)$ 
23: return( $rMap$ )

```

value higher than the utility threshold and the construction of the single cardinality residue maps takes $O(m)$ time. The single cardinality residue maps are sorted using the merge sort algorithm to construct the master map, which has the time complexity of $O(m * \text{Log } m)$.

2. *Recursive generation of residue maps of higher cardinality:*

The residue maps of cardinality more than one are formed by performing the join operations. Based on Condition-1, for a given residue value, an average of $O(m/2)$ comparisons are performed. Since the join operations are directly performed on the bit-vectors, the time complexity for generating m residue maps is $O(m * (m/2)) \approx O(m^2)$.

Thus, the total execution time for R-Miner can be represented as

$$O(n) + O(m) + O(m * \text{Log}(m)) + O(m^2) \approx O(n) + O(m^2)$$

4.4 Memory requirement analysis

For a database with m distinct items, a maximum of m residue maps of cardinality one will be constructed from the database scan. Since the utility value is distributed in a log normal fashion among n transactions, the memory required by each residue map is estimated to be $O(\log(n))$. Thus, the total memory requirement for m residue maps is $O(m \log(n))$. Additionally, the master map requires a memory of $O(m)$ as it stores m pointers to m residue maps.

For generating the residue maps of higher cardinality from the single cardinality residue maps, if $m/2$ successful join operations are performed on an average, then the memory required for generating higher cardinality residue maps is $O(m^2)$.

Thus, the total memory requirement for R-Miner can be represented as

$$O(m \log(n)) + O(m) + O(m^2) \approx O(m^2)$$

4.5 R-Miner: illustration using an example

In this section, we discuss R-Miner algorithm step by step with a running example. Consider the database provided in Fig. 10 that consists of six transactions and seven distinct items.

The individual steps of running R-Miner algorithm on the input database for a minimum utility threshold of 25 are as listed below.

TID	Transaction	Transaction Utility
1	3:1, 5:3, 1:5, 2:10, 4:6, 6:5	30
2	5:3, 2:8, 4:6	17
3	3:1, 1:5, 4:2	8
4	3:6, 5:6, 1:10	22
5	3:6, 5:3, 2:4	13
6	5:6, 2:8	14

Fig. 10 Dataset of sample Transactions

Item	TID-list	Total Utility	Residue
3	1:1	1	24
5	1:3	3	22
1	1:5	5	20
2	1:10	10	15
4	1:6	6	19
6	1:5	5	20

Fig. 11 Set of Residue maps after the scan of first transaction in the database

Item	TID-list	Total Utility	Residue
3	1:1, 3:1, 4:6, 5:6	14	11
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
1	1:5, 3:5, 4:10	20	5
2	1:10, 2:8, 5:4, 6:8	30	-5
4	1:6, 2:6, 3:2	14	11
6	1:5	5	20

Fig. 12 Set of Residue maps after the first scan of the database

Item	TWU
1	60
2	74
3	73
4	55
5	96
6	30

Fig. 13 Transaction Weighted utilisation of Itemsets

ItemSet		Item	TID-list	Total Utility	Residue
6	●	3	1:1, 3:1, 4:6, 5:6	14	11
3	●	5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
4	●	1	1:5, 3:5, 4:10	20	5
1	●	2	1:10, 2:8, 5:4, 6:8	30	-5
5	●	4	1:6, 2:6, 3:2	14	11
2	●	6	1:5	5	20

Fig. 14 Master map based on residue maps in Fig. 12

- Step 1 Construction of residue maps of cardinality one:** In this step, each transaction in the input database is scanned and the residue maps are constructed for every distinct item. The first transaction consists of six different itemsets that leads to the formation of the residue maps as shown in Fig. 11. The transaction weighted utilisation (TWU) of each itemset is also computed, as shown in Fig. 13. Similarly, the residue maps are updated for every subsequent transaction in the database. The final set of residue maps, when all the transactions have been scanned, is shown in Fig. 12.
- Step 2 Construction of master map based on sorted order of residue maps:** The master map is formed by using the pointers to the residue maps that are sorted in increasing order of total utility value. The representation of the master map is provided in Fig. 14. For constructing the master map, the itemsets having the transaction weighted utilisation value higher than the minimum utility threshold are considered.
- Step 3 Generation of higher cardinality residue maps and extraction of high-utility itemsets:** The set of high-utility itemsets are extracted by using the master map, as discussed below:
 - Generation of itemsets with cardinality one:** Since the master-map contains references to the residue-maps with cardinality one, the HUIs of size one can be directly produced as an output. For instance, from Fig. 14, it can be observed that only the residue map RM_2 has the total utility value of 30, which is higher than the input threshold value of 25. Hence, the item 2 is directly produced as an output from the master-map.
 - Generation of itemsets with cardinality two:** Since the master-map contains residue maps of cardinality one only, in order to generate the residue maps with cardinality two, the join operation is performed as follows. For each element in the master map, the residue value is compared against the total utility value of the last element in the master-map. If the Condition-1 is satisfied, perform the join operation and continue traversing the master map until a residue-map is encountered such that the total utility

is less than the residue. When the join operation is performed, a new residue-map of cardinality two is constructed and the total utility value is checked against the minimum utility threshold. If the total utility value is higher than the threshold, then that itemset is produced as an output in this step. The sequence of comparisons for the master map to produce the residue maps of cardinality two is shown in Figs. 15, 16, 17, 18 and 19.

For example, in order to perform join operation for RM_6 , the residue value of 20 is compared against the total utility value from the last element in the master-map as shown in Fig. 15, until a master map occurs such that the residue value is higher than the total utility. In this case, the total utility of RM_4 is 14, which is less than the residue value of 20, and hence, no further comparisons and join operations are performed. Thus, based on the join operations, the following set of residue maps will be generated: $RM_{2,6}$, $RM_{5,6}$ and $RM_{1,6}$.

Similar procedure is followed for generating the remaining residue maps: RM_3 to RM_5 , which is depicted in Figs. 16, 17, 18 and 19, respectively. An important point to be noted while doing the comparisons for performing the join operations is that the join operation must be performed only with the residue maps below the current residue map. This is intuitive since the join operation of a map with itself will be meaningless, and the join operations of a map with the residue-maps below its index will have the total utility value lower than the required residue. Thus, after this step, new residue maps of cardinality two are generated based on the valid set of comparisons.

- c. *Generation of itemsets with cardinality greater than two:* Residue maps of cardinality n are formed using the lower cardinality residue maps, as follows. For the formation of a residue map having cardinality n , a join operation is performed between the residue maps of cardinality $n-1$ and 1. Since the master map contains the set of residue maps of cardinality one, to generate the itemset of length n , the master map is traversed and the join operation is performed till the total utility of the residue map in the master map is higher or equal to the residue of the residue map with cardinality $n-1$. This procedure is repeated recursively for the newly formed residue map. If the computed total utility is higher than the user specified threshold, then the itemset will be produced as an output.

The itemsets with cardinality of three are formed by performing the join operation of the residue maps of cardinality two with the residue maps of cardinality one. For instance, $RM_{2,6}$ is produced as the result of the join operation from Fig. 15. The residue from $RM_{2,6}$ is then compared with the total utility values in the master map as shown in Fig. 20 that results in the formation of the residue map of size three. Similar procedure is followed for the resultant residue map as depicted in Figs. 21, 22, 23 and 24.

5 Experimental evaluation

The R-Miner algorithm is implemented in Java on a computer with 1.4 GHz Quad-Core Intel i5 processor and 8 GB of memory. In order to test the efficiency of the proposed algorithm, an extensive set of experiments are performed on various datasets and its performance is compared with the state-of-the-art mining algorithms. The following set of experiments are designed for a detailed performance analysis of R-Miner:

Item	TID-list	Total Utility	Residue
⑥	1:5	5	20
3	1:1, 3:1, 4:6, 5:6	14	11
4	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

----->

Item	TID-list	Total Utility	Residue
⑥	1:5	5	20
3	1:1, 3:1, 4:6, 5:6	14	11
4	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

⋮

Item	TID-list	Total Utility	Residue
⑥	1:5	5	20
3	1:1, 3:1, 4:6, 5:6	14	11
4	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

←-----

Item	TID-list	Total Utility	Residue
⑥	1:5	5	20
3	1:1, 3:1, 4:6, 5:6	14	11
4	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

Fig. 15 Set of join operations for RM_6

Item	TID-list	Total Utility	Residue
6	1:5	5	20
③	1:1, 3:1, 4:6, 5:6	14	11
4	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

----->

Item	TID-list	Total Utility	Residue
6	1:5	5	20
③	1:1, 3:1, 4:6, 5:6	14	11
4	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

⋮

Item	TID-list	Total Utility	Residue
6	1:5	5	20
③	1:1, 3:1, 4:6, 5:6	14	11
4	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

←-----

Item	TID-list	Total Utility	Residue
6	1:5	5	20
③	1:1, 3:1, 4:6, 5:6	14	11
4	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

Fig. 16 Set of join operations for RM_3

Item	TID-list	Total Utility	Residue
6	1:5	5	20
3	1:1, 3:1, 4:6, 5:6	14	11
④	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

----->

Item	TID-list	Total Utility	Residue
6	1:5	5	20
3	1:1, 3:1, 4:6, 5:6	14	11
④	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

⋮

Item	TID-list	Total Utility	Residue
6	1:5	5	20
3	1:1, 3:1, 4:6, 5:6	14	11
④	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

←-----

Item	TID-list	Total Utility	Residue
6	1:5	5	20
3	1:1, 3:1, 4:6, 5:6	14	11
④	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

Fig. 17 Set of join operations for RM_4

Item	TID-list	Total Utility	Residue
6	1:5	5	20
3	1:1, 3:1, 4:6, 5:6	14	11
4	1:6, 2:6, 3:2	14	11
①	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

----->

Item	TID-list	Total Utility	Residue
6	1:5	5	20
3	1:1, 3:1, 4:6, 5:6	14	11
4	1:6, 2:6, 3:2	14	11
①	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

Fig. 18 Set of join operations for RM_1

Item	TID-list	Total Utility	Residue
6	1:5	5	20
3	1:1, 3:1, 4:6, 5:6	14	11
4	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
⑤	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

Fig. 19 Set of join operations for RM_5

Item	TID-list	Total Utility	Residue
6	1:5	5	20
3	1:1, 3:1, 4:6, 5:6	14	11
4	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

⋮

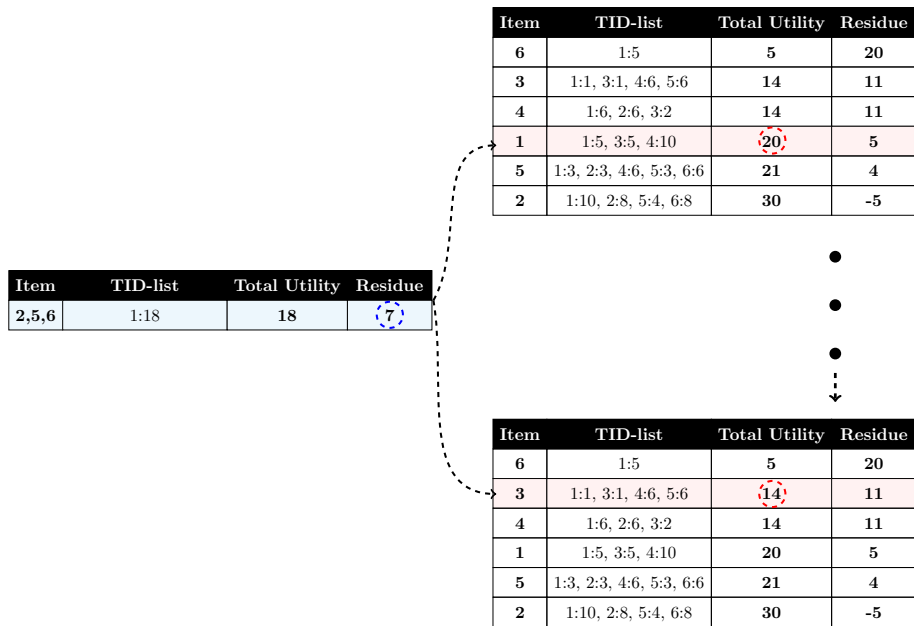
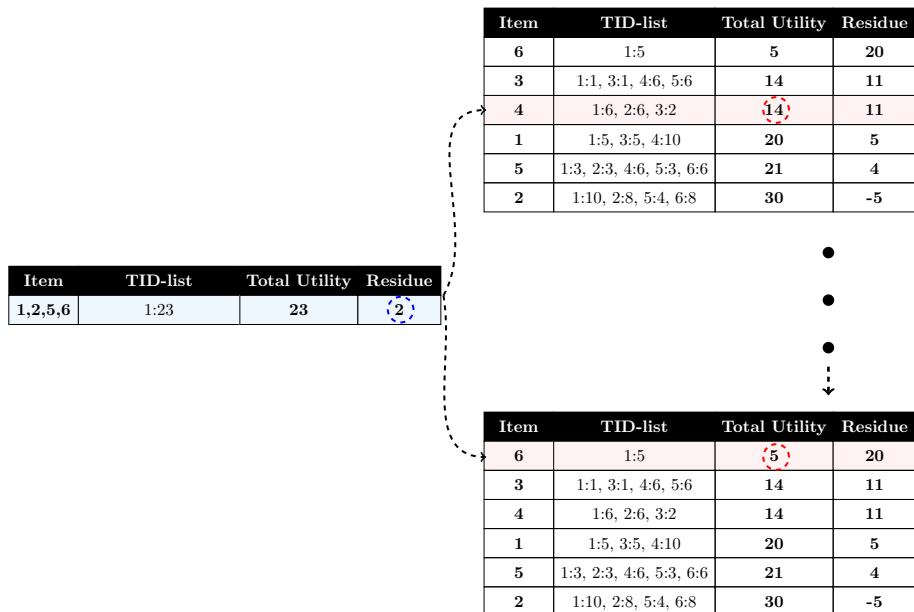
Item	TID-list	Total Utility	Residue
2,6	1:15	15	10

⋮

Item	TID-list	Total Utility	Residue
6	1:5	5	20
3	1:1, 3:1, 4:6, 5:6	14	11
4	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

Fig. 20 Set of join operations for $RM_{2,6}$

1. Two ordering schemes for the master map are explored and compared in terms of the execution time. These are, (i) total utility based on ascending order and (ii) total utility based on descending order.
2. The total execution time (in sec) of R-Miner algorithm is compared with other mining algorithms.
3. The memory consumption (in MB) of R-Miner algorithm is compared with other mining algorithms.
4. The total number of join operations required by R-Miner algorithm is compared with two other algorithms that also use similar join operations to generate the candidates.

Fig. 21 Set of join operations for $RM_{2,5,6}$ Fig. 22 Set of join operations for $RM_{1,2,5,6}$

Item	TID-list	Total Utility	Residue
1,2,4,5,6	1:29	29	-4

Item	TID-list	Total Utility	Residue
6	1:5	5	20
3	1:1, 3:1, 4:6, 5:6	14	11
4	1:6, 2:6, 3:2	14	11
1	1:5, 3:5, 4:10	20	5
5	1:3, 2:3, 4:6, 5:3, 6:6	21	4
2	1:10, 2:8, 5:4, 6:8	30	-5

Fig. 23 Set of join operations for $RM_{1,2,4,5,6}$

Item	TID-list	Total Utility	Residue
1,2,3,4,5,6	1:30	30	-5

Fig. 24 Set of join operations for $RM_{1,2,3,4,5,6}$

5.1 Ordering schemes for the master map

The comparison results of the ordering schemes for forming the master map are shown in Fig. 25. It can be observed from this figure that the performance of R-Miner is better when the master map is sorted in the ascending order of the total utility values. The reason for this is attributed to the fact that when the residue maps are sorted in descending order of the total utility values, additional time is incurred in comparison with the residue maps that have lower total utility value than the residue of the current map. Thus, the remaining experiments are performed by using the total utility based on ascending ordering scheme for the master map.

5.2 Execution time

The total execution time and memory consumption time of R-Miner algorithm are compared with the benchmark algorithms: ULB-Miner [5], FHM [7], H-Miner [13], HUI-Miner [16], EFIM [28]. The key highlights of these algorithms are discussed below:

1. HUI-Miner [16]: The HUI-Miner algorithm uses utility-list, which is used to store utility related information. In order to mine high-utility item-sets, HUI-Miner avoids the generation of candidate item-sets and their utility computation, unlike the traditional high-utility mining algorithms [18].
2. FHM [7]: The FHM algorithm improves the computation of the costly join operation of the HUI-Miner [16] algorithm, by using a novel structure called the EUCS (Estimated co-occurrence utility structure) and is up to 6 times faster than HUI-Miner.
3. EFIM [28]: The EFIM (Efficient high-utility Itemset Mining) algorithm introduces two upper-bounds called the sub-tree utility and local utility to effectively prune the search space for high-utility item-set mining. EFIM improves the performance by up to two orders as compared to HUI-Miner and FHM.
4. H-Miner [13]: The H-Miner algorithm uses a compact utility list and virtual hyperlink data structure for storing itemset information and improves performance by up to three orders as compared to EFIM [28].

For the purpose of experimental evaluation, both real world as well as synthetic datasets are used as provided in Table 1. Experimental results indicate that, for large data-sets like *kosark*, the R-Miner algorithm can improve the performance by up to two times. The R-

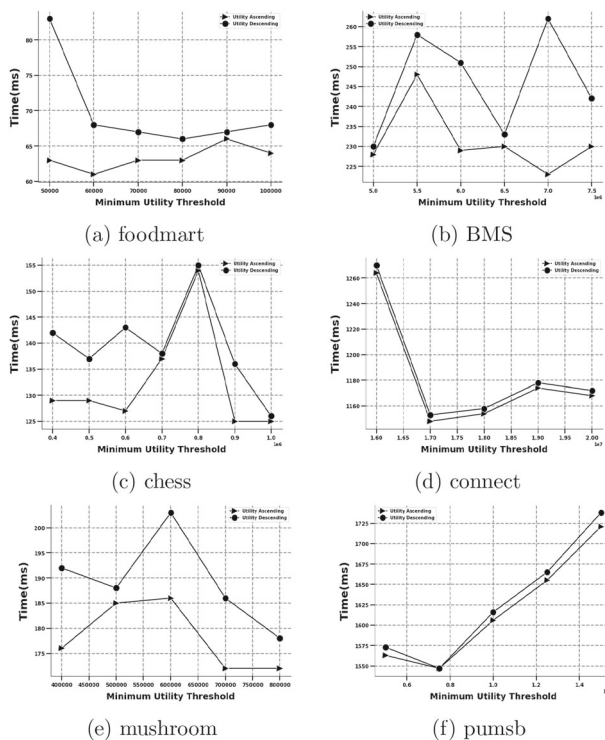


Fig. 25 Ordering schemes for master map

Table 1 Statistics of datasets

Data-Set	Transactions	Items	Average Length
<i>BMS</i>	77,512	3,340	4.62
<i>Chess</i>	3,196	75	37
<i>Connect</i>	67,557	129	43.0
<i>Kosark</i>	990,000	41,270	8.1
<i>Pumsb</i>	49,046	2,113	74

Miner algorithm also works well on dense data-sets like *chess* and *pumsb*. These datasets are available at SPMF library [6].

The comparison results of the total execution time by varying the minimum utility threshold values for each dataset are provided in Fig. 26. It can be observed from this figure that for almost all the datasets, R-Miner has better execution time. The improved performance is attributed to the controlled expansion of the search space by the R-Miner algorithm. This is done by limiting the number of join operation and the count of residue-maps, for producing the final set of HUIs.

For the *pumsb* dataset, the graphs of some algorithms are partially missing in Figs. 26 and 27, since the algorithms did not finish executing on some values of the minimum utility threshold.

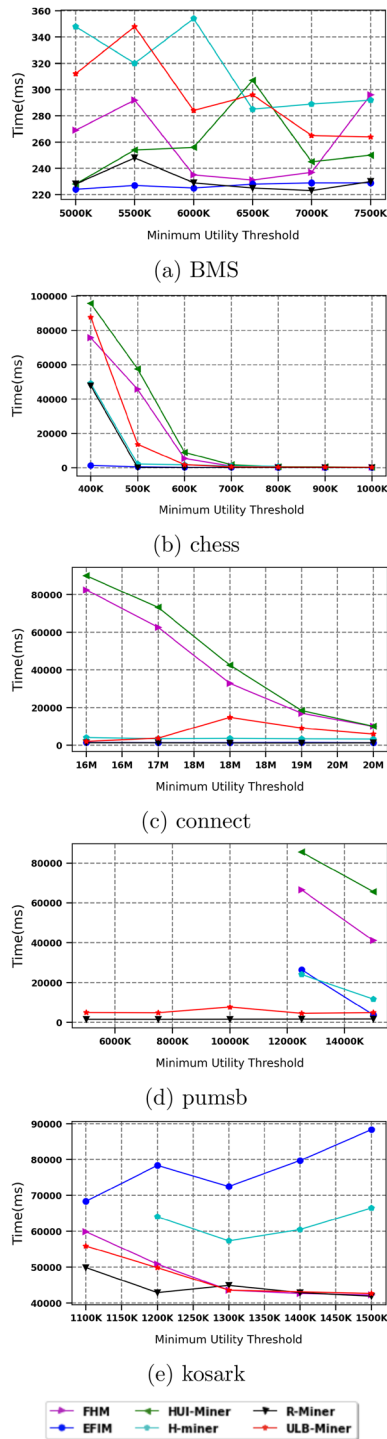
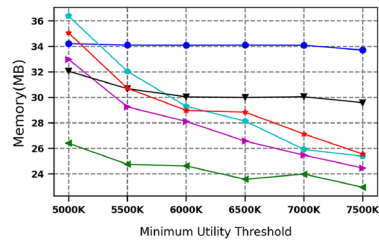
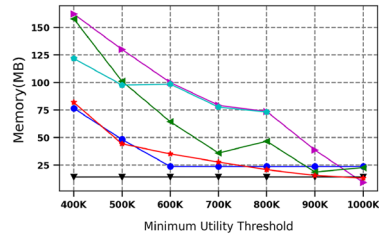


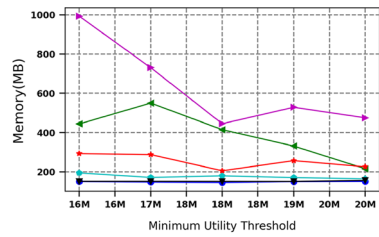
Fig. 26 Execution Time



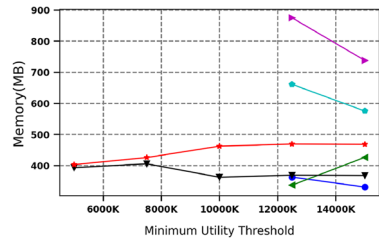
(a) BMS



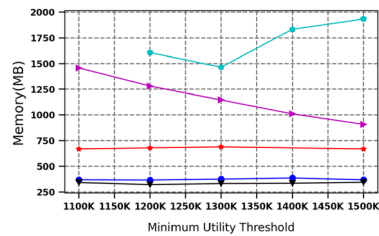
(b) chess



(c) connect



(d) pumsb



(e) kosark



Fig. 27 Memory Consumption

Table 2 Comparison of count of the Join Operations

DataSet	Min Utility	Join Count		
		R-Miner	HUI-Miner	FHM
<i>foodmart</i>	10,000	90,852	1,220,530	1,211,093
	15,000	10,906	1,175,817	1,175,817
	20,000	1402	1,072,665	1,072,665
	25,000	249	814,134	814,134
<i>BMS</i>	2,500,000	7797	12,126	10,135
	2,700,000	3179	4530	4530
	2,900,000	1292	1292	1292
<i>Chess</i>	70,000	36,041	–	–
	75,000	5500	–	–
	80,000	1017	–	–
	85,000	1017	–	–
<i>Connect</i>	2,500,000	585	–	–
	2,600,000	253	–	–
	2,700,000	250	–	–
	2,800,000	244	–	–
	2,900,000	244	14	10

5.2.1 Memory consumption

The comparison of the memory consumed by varying the minimum utility threshold values for each dataset is provided in Fig. 27.

The main source of memory consumption of R-Miner is the construction of residue maps. The higher the number of residue maps generated, the higher is the memory consumption. The generation of a large number of residue-maps is limited by the conditional join operations in R-miner, which leads to memory optimisation. From Fig. 27, it can also be observed that the memory consumption of the R-Miner algorithm is comparable with that of the other algorithms.

5.3 Comparison of the number of join operations

The HUI-Miner [16] and FHM [7] algorithms generate the potential candidates for mining HUIs by performing an intersection between the linked-list like structure, called utility list. In this section, the number of join operations performed by three algorithms are compared and the results are provided in Table 2. It can be observed from this table that, for all the cases, R-Miner is the clear winner. It may be noted that columns marked with a blank in Table 2, are for the cases when the algorithms did not finish execution after a run duration of more than 25,000 s.

6 Conclusion and future work

In this paper, we have undertaken the design challenge for the use of efficient data-structures for utility mining to address the drawbacks of the existing utility-list-based techniques. We

have discussed a new algorithm, named as R-Miner, to extract the high utility items from a dataset. The R-Miner algorithm is designed primarily based on two novel data structures: residue map and master map. The residue-maps are based on the novel concept of residue that uses the information of the minimum utility threshold. The master map is formed based on the residue maps of single itemsets and it stores the utility relevant information. The master map is traversed recursively to generate the search space for high-utility itemsets. Several experiments are conducted on both real and synthetic datasets to assess the performance of R-Miner algorithm. The R-Miner has been found efficient in terms of the execution time as well as the memory, especially for the datasets with less number of distinct items. We have addressed the major research gap of designing the novel and efficient data-structures for utility mining.

We will extend the proposed work for designing the algorithms for incremental datasets and datasets with negative utility values. Since there exist several variations of high-utility mining, the concept of residue used in R-Miner can also be explored for other areas of pattern mining like on-shelf high-utility mining, dynamic datasets, and mining with various discount strategies.

Author Contributions Study conception and design of the R-Miner algorithm were carried out by both authors. The figures and manuscript preparation was carried out by Pushp. Prof Satish Chand undertook the critical revision of the article. Both authors reviewed the results and approved the final version of the manuscript.

Declarations

Conflict of interest The authors declare no competing interests.

References

1. Agrawal R, Imieliński T, Swami A (1993) Mining association rules between sets of items in large databases. In: ACM sigmod record, vol 22. ACM, pp 207–216
2. Ahmed CF, Tanbeer SK, Jeong B-S, Lee Y-K (2009) Efficient tree structures for high utility pattern mining in incremental databases. *IEEE Trans Knowl Data Eng* 21(12):1708–1721
3. Baek Yoonji, Yun Unil, Kim Heonho, Kim Jongseong, Vo Bay, Truong Tin, Deng Zhi-Hong (2021) Approximate high utility itemset mining in noisy environments. *Knowl-Based Syst* 212:106596
4. Chand S et al. (2021) Mining of high utility itemsets for incremental datasets. In: 2021 International conference on electrical, computer, communications and mechatronics engineering (ICECCME). IEEE, pp 01–06
5. Duong Quang-Huy, Fournier-Viger Philippe, Ramampiaro Heri, Nørnvåg Kjetil, Dam Thu-Lan (2018) Efficient high utility itemset mining using buffered utility-lists. *Appl Intell* 48(7):1859–1877
6. Fournier-Viger P, Lin JC-W, Gomariz A, Gueniche T, Soltani A, Deng Z, Lam HT (2016) The spmf open-source data mining library version 2. In: Proceedings of the Machine learning and knowledge discovery in databases: European Conference, ECML PKDD 2016, Riva del Garda, Italy, September 19–23, 2016, Part III 16. Springer, pp 36–40
7. Fournier-Viger P, Wu C-W, Zida S, Tseng VS (2014) FHM: faster high-utility itemset mining using estimated utility co-occurrence pruning. In: International symposium on methodologies for intelligent systems. Springer, pp 83–92
8. Gan W, Lin JC-W, Fournier-Viger P, Chao H-C, Philip SY (2016) HUOPM: high-utility occupancy pattern mining. *IEEE Trans Cybern* 50(3):1195–1208
9. Han J, Pei J, Yin Y (2000) Mining frequent patterns without candidate generation. In: ACM sigmod record, vol 29. ACM, pp 1–12
10. Kim H, Ryu T, Lee C, Kim C, Yoon E, Vo B, Lin JC-W, Yun U (2022) EHMIN: efficient approach of list based high-utility pattern mining with negative unit profits. *Expert Syst Appl* 209:118214
11. Kim H, Yun U, Baek Y, Kim H, Nam H, Lin JC-W, Fournier-Viger P (2021) Damped sliding based utility oriented pattern mining over stream data. *Knowl-Based Syst* 213:106653

12. Krishnamoorthy Srikumar (2015) Pruning strategies for mining high utility itemsets. *Expert Syst Appl* 42(5):2371–2381
13. Krishnamoorthy Srikumar (2017) HMiner: efficiently mining high utility itemsets. *Expert Syst Appl* 90:168–183
14. Lee C, Ryu T, Kim H, Kim H, Vo B, Lin JC-W, Yun U (2022) Efficient approach of sliding window-based high average-utility pattern mining with list structures. *Knowl-Based Syst* 256:109702
15. Lin JC-W, Djenouri Y, Srivastava G, Fourier-Viger P (2022) Efficient evolutionary computation model of closed high-utility itemset mining. *Appl Intell* 1–13
16. Liu M, Qu J (2012) Mining high utility itemsets without candidate generation. In: *Proceedings of the 21st ACM international conference on Information and knowledge management*. ACM, pp 55–64
17. Liu Y, Liao W-k, Choudhary A (2005) A fast high utility itemsets mining algorithm. In: *Proceedings of the 1st international workshop on Utility-based data mining*. ACM, pp 90–99
18. Liu Y, Liao W-K, Choudhary A (2005) A two-phase algorithm for fast discovery of high utility itemsets. In: *Proceedings of the Advances in knowledge discovery and data mining: 9th Pacific-Asia conference, PAKDD 2005, Hanoi, Vietnam, May 18–20, 2005*. Springer, pp 689–695
19. Nam Hyojun, Yun Unil, Vo Bay, Truong Tin, Deng Zhi-Hong, Yoon Eunuchul (2020) Efficient approach for damped window-based high utility pattern mining with list structure. *IEEE Access* 8:50958–50968
20. Ryu T, Yun U, Lee C, Lin JC-W, Pedrycz W (2022) Occupancy-based utility pattern mining in dynamic environments of intelligent systems. *Int J Intell Syst* 37(9):5477–5507
21. Tseng VS, Shie B-E, Wu C-W, Philip SY (2013) Efficient algorithms for mining high utility itemsets from transactional databases. *IEEE Trans Knowl Data Eng* 25(8):1772–1786
22. Tseng VS, Wu VS, Shie B-E, Yu PS (2010) Up-growth: an efficient algorithm for high utility itemset mining. In: *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*, pp 253–262
23. Wu JM-T, Li Z, Srivastava G, Yun U, Lin JC-W (2021) Analytics of high average-utility patterns in the industrial internet of things. *Appl Intell* 1–14
24. Yao H, Hamilton HJ (2006) Mining itemset utilities from transaction databases. *Data Knowl Eng* 59(3):603–626
25. Yao H, Hamilton HJ, Butz CJ (2004) A foundational approach to mining itemset utilities from databases. In: *Proceedings of the 2004 SIAM international conference on data mining*. SIAM, pp 482–486
26. Lele Yu, Gan Wensheng, Chen Zhixiong, Liu Yining (2023) IDHUP: incremental discovery of high utility pattern. *J Internet Technol* 24(1):135–147
27. Yun Unil, Kim Heonho, Ryu Taewoong, Baek Yoonji, Nam Hyojun, Lee Judae, Vo Bay, Pedrycz Witold (2021) Prelarge-based utility-oriented data analytics for transaction modifications in internet of things. *IEEE Internet Things J* 8(24):17333–17344
28. Zida S, Fournier-Viger P, Lin JC-W, Wu C-W, Tseng VS (2017) EFIM: a fast and memory efficient algorithm for high-utility itemset mining. *Knowl Inf Syst* 51(2):595–625

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Reproduced with permission of copyright owner.
Further reproduction prohibited without permission.